Topoloka: Monocular Multi-Room 3D Gaussian Splatting & Metric Indoor Navigation

Techfest 2026–27, IIT Bombay — Namma Space Round 2 Technical Abstract & SOP

Submission Field	Details
Author / Lead	Shashank Dev (`coderperson043@gmail.com`)
Team / Competition ID	`TM-60A7F03511D` · `NAMM-B3B981411E93`
Interactive Web Viewer	https://sciencebanda09.github.io/topoloka/
Source Repository	https://github.com/sciencebanda09/topoloka (MIT License)
Raw Capture & Narrated Video	`[To be updated with self-captured raw video & human-narrated walkthrough links]`

1. Problem & Objectives

Outdoor environments rely on GPS and street-view imagery, yet indoor venues (campuses, hostels, transit corridors, retail halls) remain unmapped for browser-based exploration. Existing indoor scanning approaches face three barriers:

1. **Hardware Cost:** Professional LiDAR scanners cost $4,000–$25,000.
2. **Payload & Latency:** Heavy 300–800 MB textured meshes stall browser downloads and mobile WebGL GPUs.
3. **Lack of Navigation Semantics:** Passive 3D viewers lack calibrated floor height, collision walls, spatial POI search, and walkable paths.

Topoloka converts a single continuous smartphone video into a **hybrid metric twin** — combining photorealistic **compressed 3D Gaussian Splats (`.ksplat`)** with a **5 cm metric 2.5D occupancy costmap** — deployed as a zero-backend static web application running at 50–60 FPS in desktop and mobile browsers.

2. Standardized Data Capture SOP (Reproducible Field Protocol)

Reconstruction fidelity and zero-floater geometry are enforced at capture time via `capture/SOP.md`:

Parameter	Specification	Engineering Rationale
Device & App	Consumer smartphone (Android/iOS, 1× wide lens; team unit: OnePlus) running native camera or **Sensor Logger** (`sensors.csv` IMU + MP4)	Single-lens intrinsics keep `pycolmap` `CameraMode.SINGLE` exact; optional IMU enables rotational blur checks.
Video Settings	**4K / 30 fps** (1080p fallback), **Landscape (16:9)**, **EIS / Stabilization OFF**, **AE & AF Locked**, Flash OFF	Disabling EIS prevents non-rigid frame warps that elevate SfM reprojection error above `1.0 px`.
Lighting	All overhead lights ON + daylight curtains open; avoid direct window/mirror glare	Uniform diffuse illumination avoids dark shadow noise and specular floaters.
Distance & Pace	Walk `0.4 m/s` (`0.3 m/s` through doorways/stairs), `1.2 – 2.0 m` **stand-off** from walls (centerline in narrow halls), max turn rate `≤ 30°/s`	Sustains `60–70%` **overlap** at 2.5 Hz keyframe sampling with low stride/pan motion blur.
Multi-Room / Floor Sequence	One unbroken take starting in largest room (`5 s` stationary start hold): **Pass 1** (`+10°` up-tilt for walls/ceilings) → **Pass 2** (`−20°` down-tilt for floors/furniture/QR anchors) → `5 s` **stationary loop closure** back at origin.	Still ends anchor SfM loop closure; dual-tilt captures both upper structure and floor collision seams.
Every-Corner & Doorway Rule	Pivot around each corner in a `1 m` arc keeping **corner seam + floor junction centered for** `≥ 3 s` across `45°–60°` angles; cross doorways straight-on with `±25°` exit pan; traverse stairwells at `0.25–0.35 m/s` keeping handrails & steps overlapping.	Eliminates blind corners and maintains continuous SfM feature tracks across rooms and floors.
Optional 2D Plan & Scale Photos	If pairing an architectural 2D plan: flat orthographic overhead photo under diffuse light (`0°` tilt, full corners + scale bar). For manual metric cross-check: 3 square photos (`±15°`) of a `≥ 2.00 m` tape measure laid across a doorway.	Enables independent sub-centimeter verification of the monocular scale factor `s`.
Automated QA Gate	`python -m namma.cli qa` verifies Laplacian sharpness `p10 ≥ 100`, registered views `≥ 60%`, reprojection `≤ 1.0 px`, EIS OFF, coverage `≥ 80%`, gyro blur `p90 ≤ 4 px`.	Rejects blurry or drifted takes before spending GPU training budget.

3. Reconstruction Pipeline & Mathematical Alignment

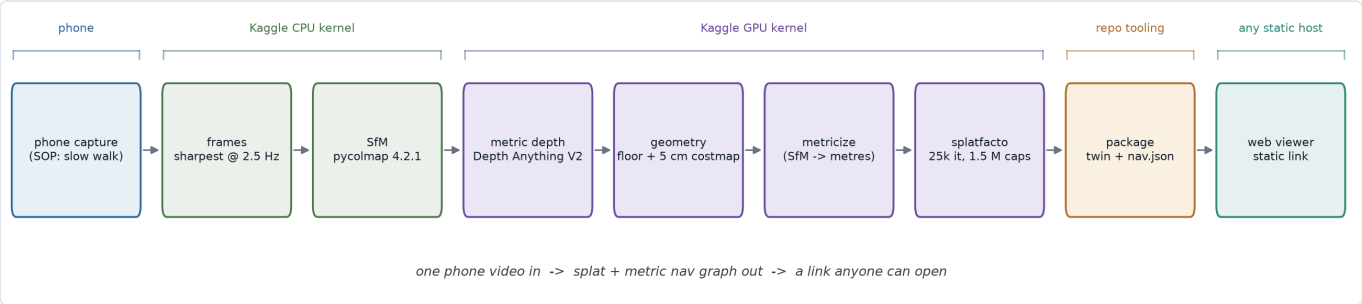


Figure 1: Modular stages from raw phone video to compressed web `.ksplat`, metric occupancy costmap, spatial POI index, and AR QR anchors.

1. **Stride-Adaptive Blur Filtering (`frames.py`):** Extracts frames at `2.5 Hz` selecting the local sharpest frame inside each stride window via Laplacian variance.
2. **Structure-from-Motion (`sfm.py`):** Incremental triangulation via `pycolmap` (`CameraMode.SINGLE`), exporting registered camera poses and sparse 3D points.
3. **Commercially Licensed Metric Depth Fusion (`depth.py`):** Estimates dense metric depth maps per registered view using `Depth-Anything-V2-Metric-Indoor-Small-hf` (**Apache 2.0**), fusing masked points within `0.15 – 12.0 m` (`3 cm` voxel grid) and solving global scale via pairwise median ratio against sparse SfM camera depth:

```
s = median( D_metric(u, v) / Z_sfm(u, v) ), p_nav = s · R^T · (p_scene − p_origin)
```

4. **Vectorized RANSAC Gravity Alignment & Costmap (`geometry.py`):** Fits the dominant horizontal floor plane with RANSAC, derives an orthonormal right-handed rotation matrix `R` (`det(R) = +1`, `floor_z = 0.0 m`), rasterizes a **5 cm 2.5D occupancy grid** (`free`, `obstacle`, `unknown`) with continuous corridor carving between camera poses, and encodes **Vectorized Clearance-Cost Weighting (VCCW)** via Euclidean Distance Transform (EDT) so planned paths naturally favor wide aisle centers.
5. **3D Gaussian Splatting & `.ksplat` Compression (`splat_convert.py`, `convert_to_ksplat.mjs`):** Trains float-pruned 3DGS (`nerfstudio splatfacto` / `gsplat`, **Apache 2.0**), strips spherical-harmonic bands above DC for compact web streaming, and packs spatial-grid sorted **Level-1** `.ksplat` (~`10–11.5 MB`, ~`75%` smaller than raw `.splat`), eliminating browser-side partitioning stalls.
6. **In-Browser Interactive Viewer (`web/src/`):** Streams an instant colored 3D `cloud.ply` preview alongside live progress telemetry (`downloaded X.X MB` → `building scene`), then swaps seamlessly into `50–60 FPS` Three.js WebGL `.ksplat` rendering with:
   - **First-person walkthrough & wall collision** (`WASD` / pointer lock / mobile touch stick) synced to the live **2D minimap (`M`)** and **view model switcher** (`Gaussian splats` / `Point cloud`).
   - **Spatial + Fuzzy POI Search & Turn-by-Turn A* Navigation:** 1 m spatial bucket + inverted token + Levenshtein index (`/`), explicit `Jump here`, `Navigate`, Pure-Pursuit lookahead + auto-rerouting, and **in-browser POI floor tagging (`E`)** with `localStorage` persistence and JSON export.
   - **Interactive Sandbox (`G`) & Orientation HUD:** Place/rotate dynamic props (`Cone`, `Barrier`, `Crate`, `Sign`) that carve live obstacles into the A* router; adjust `rx/ry/rz/oy` or `Flip 180°` into shareable URLs.
   - **QR Anchor AR Mode:** Relocalizes onto `anchors.json` QR fiducials (`namma anchors`) and displays directional AR turn guidance.

4. Team Capture & Verification Slot

Self-captured multi-room dataset metrics (`my_space`, following Section 2 SOP) will be populated here once reconstruction completes:

Verification Metric	Target Gate	Measured (`my_space`)
Raw Capture & Rooms Covered	Multi-room continuous loop (`≥ 90 s`, EIS OFF)	`[Pending user video]`
SfM Registration / Reproj. Error	`≥ 60%` registered · `≤ 1.0 px` median	`[From capture_qa.json]`
Metric Scale MAD	`≤ 10%` median absolute deviation	`[From depth_report.json]`
Compressed `.ksplat` Payload & FPS	`≤ 15 MB` `.ksplat` · `≥ 50 FPS` WebGL	`[From meta.json + Chrome E2E]`

5. Third-Party Tools, Services & Licensing Compliance

All pipeline models, libraries, and frontend engines permit **commercial use** under permissive open-source licenses; no proprietary/non-commercial (`CC-BY-NC`) model weights are used in production:

Component	Purpose	License	Commercial Use
pycolmap (`colmap/pycolmap`)	Incremental Structure-from-Motion & camera poses	BSD 3-Clause	Permitted
Depth Anything V2 Metric Indoor Small	Monocular metric depth & scale anchor (`depth.py`)	Apache 2.0	Permitted
Nerfstudio (`splatfacto`) & `gsplat`	CUDA-accelerated 3D Gaussian Splatting training	Apache 2.0	Permitted
OpenCV (`opencv-python-headless`) & NumPy / SciPy	Sharpness QA, RANSAC alignment, EDT costmap carving	Apache 2.0 / BSD	Permitted
Three.js & `@mkkellogg/gaussian-splats-3d`	WebGL rendering, `.ksplat` decompression, first-person HUD	MIT	Permitted
Vite / TypeScript / Vitest / Pytest	Frontend bundling, typechecking, unit tests	MIT	Permitted
GitHub Pages & Kaggle Compute (Build-Time)	Static CDN hosting & build-time T4 training (`$0` runtime API)	Service Terms	Static delivery free

6. Third-Party API vs. Own GPU Infrastructure Cost Analysis

Because Topoloka performs **100% of reconstruction on open-source weights and delivers static `.ksplat` files**, inference incurs **$0 per-view or per-API-call runtime cost**. Below is a unit-economics comparison for processing 3D indoor spaces at increasing scale:

Deployment Model	Compute Unit & Assumptions	Cost / Scene	100 Scenes	1,000 Scenes	Notes / Scalability
Commercial 3D APIs *(SaaS scan/photogrammetry APIs)*	Typical managed processing (`$2.00–$5.00`/scan) + hosted viewer (`$49–$99/mo` seat/storage)	**`$2.50 – $5.50`**	`$840+` *(incl. 1 yr SaaS)*	`$3,200 – $6,000+`	Per-minute/per-scan billing scales linearly; vendor lock-in & data egress limits.
Self-Hosted Cloud GPU *(On-demand NVIDIA L4 `$0.75/hr` or T4 `$0.35/hr`)*	~25 min L4 (`$0.31`) or ~40 min T4 (`$0.23`) per multi-room `.ksplat` + static GitHub/S3 CDN (`~$0.01/scene/mo`)	**`$0.25 – $0.35`**	**`~$31`**	**`~$320`**	Auto-scales horizontally on spot GPUs; **~88% cheaper** than managed APIs with zero runtime server cost.
Owned GPU Workstation *(e.g. RTX 4070 `$600` amortized over 3 yrs + `$0.12/kWh` power)*	~15 min @ 220W (`$0.007` electricity/scene) + hardware capex (`$600`)	**`$0.01` marginal** (`$0.61` @ 1k)	`$607` *(incl. GPU capex)*	**`$607` total** (`$0.61/scene`)	Breaks even against `$3/scene` commercial APIs at **~200 scenes**; ideal for offline privacy-sensitive venues.
Competition Tier (Current)	Free Kaggle T4 GPU quota (`30 hr/wk`) + static GitHub Pages	**`$0.00`**	`$0.00` *(batched)*	N/A *(quota bound)*	Zero spend during prototyping; seamless migration to self-hosted L4 workers for commercial load.

Takeaway: Running open Apache-2.0 / MIT models on owned or spot L4 GPUs paired with static `.ksplat` CDN delivery eliminates per-visitor API fees completely and achieves **~8–12× lower cost per reconstructed space** above 200 captures compared to commercial reconstruction APIs.